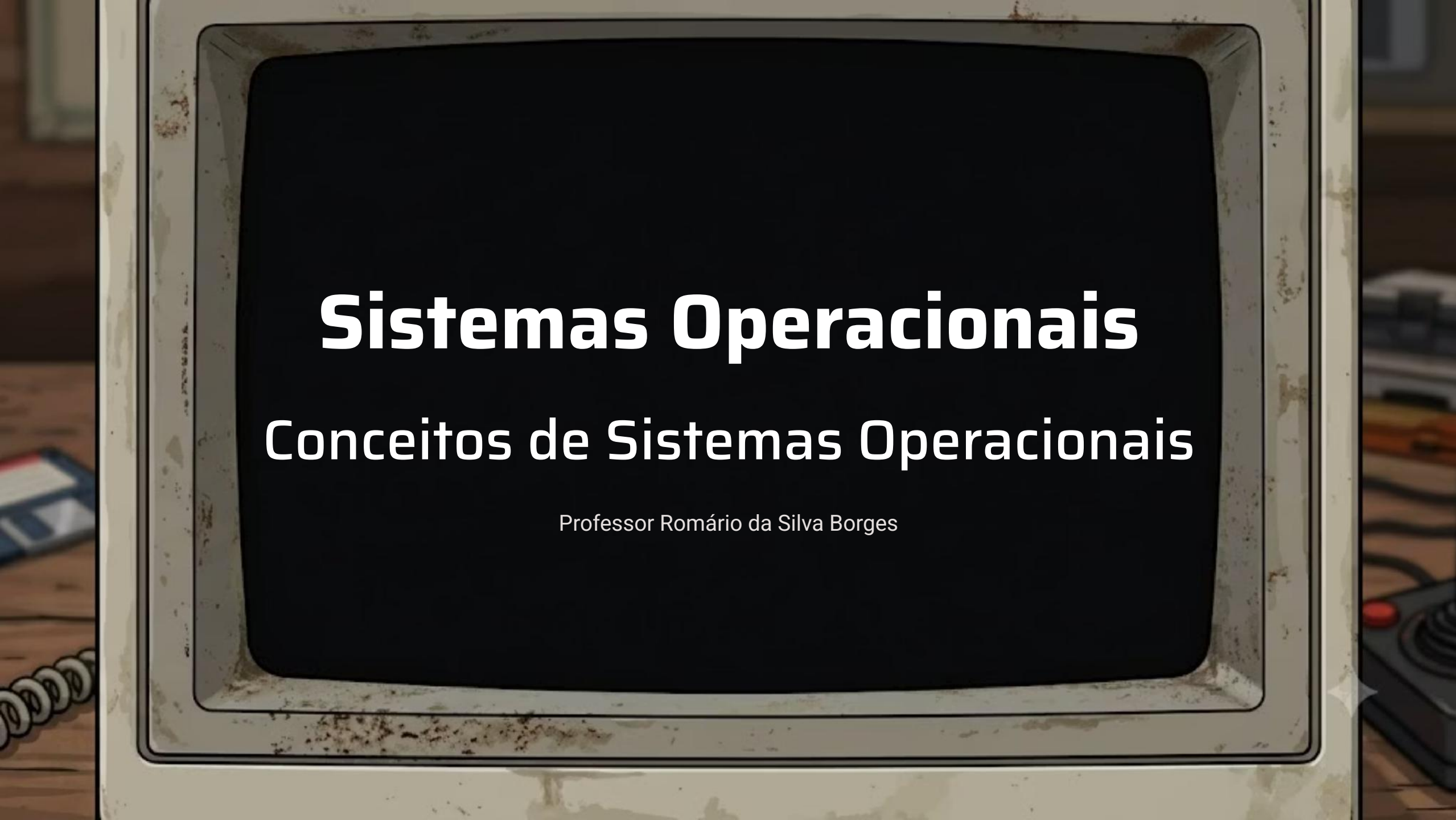




Sistemas Operacionais

Conceitos de Sistemas Operacionais

Professor Romário da Silva Borges

O que veremos nesta aula?

1 Diferentes SOs

Diferentes tipos de sistemas operacionais e seus contextos de uso.

2 Conceitos Fundamentais

Processos, arquivos, proteção, chamadas de sistema.

3 Chamadas de Sistema

API, biblioteca e kernel: a ponte entre programas e hardware.

4 Estruturas de SOs

"Se dois sistemas operacionais executam os mesmos aplicativos básicos, acessam arquivos e controlam memória, **por que eles podem ser tão diferentes por dentro?**"

Um Sistema de Computação

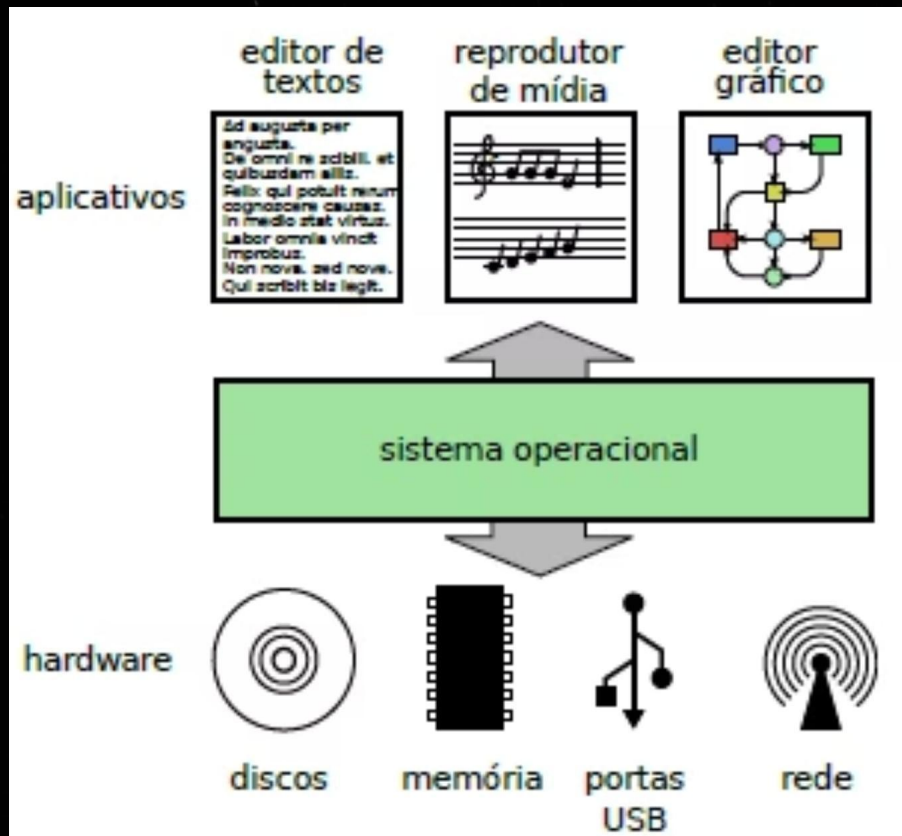
Um sistema de computação pode ser dividido em quatro componentes essenciais:



O **sistema operacional** é o componente central que gerencia esses recursos, tornando-o indispensável para o funcionamento de praticamente todo sistema de computação moderno.

O que é um SO ?





O que é um SO?

1. O sistema operacional atua **como intermediário entre o usuário, os programas e o hardware**.
2. Seu principal objetivo é permitir que programas sejam executados de forma conveniente e eficiente.
3. O sistema operacional também funciona como **gerenciador de recursos**.
4. Além de facilitar o uso do computador, o sistema operacional deve garantir a operação correta, segura e organizada do sistema de computação.

O que um sistema operacional faz?

Um sistema operacional pode ser entendido por duas grandes perspectivas:

Máquina Estendida

- Esconde detalhes complexos do hardware
- Oferece abstrações mais simples para os programas

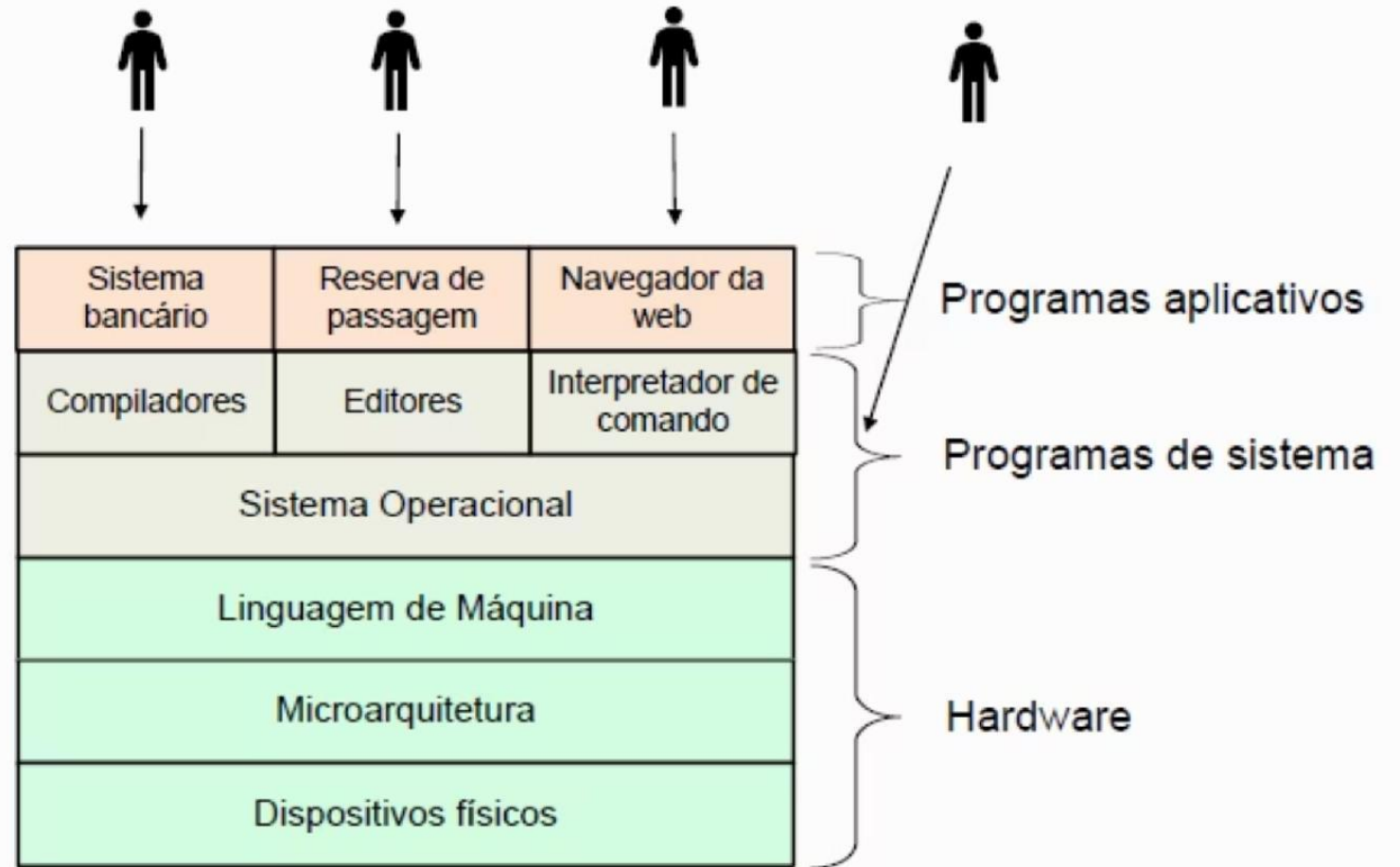
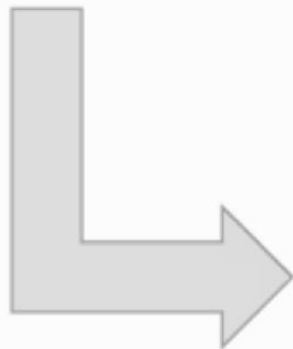
Gerenciador de Recursos

- Controla CPU, memória, dispositivos e arquivos
- Organiza o uso dos recursos entre programas e usuários

Para facilitar a programação, os SOs fornecem serviços aos programas e aos usuários destes programas. Tais serviços variam de SO para SO.

Estrutura dos computadores

Segundo
Andrew
Tanenbaum



Imagine, se:

Cada programador tivesse de preocupar-se com o modo como as unidades de disco funcionam e com todas as dúzias de coisas que poderiam dar errado ao ler um bloco de disco?

Cada programador tivesse de preocupar-se com todos os outros processos que rodam simultaneamente no sistema operacional?

Cada programador tivesse que se preocupar com o modelo do processador utilizado?

- ① “ Há muitos anos tornou-se bastante evidente a necessidade de encontrar uma maneira de isolar os programadores da complexidade do hardware . A maneira com que isso se desenvolveu gradualmente foi colocar uma camada de software por cima do hardware básico para gerenciar todas as partes do sistema.”

Por que estudar Sistemas Operacionais?

- 1 Ajudará na utilização de diversos sistemas;
- 2 Ajudará na programação de diversos sistemas;
- 3 Ajudará a projetar aplicações eficazes;
- 4 Ajudará a projetar aplicações eficientes.

 Multidisciplinaridade : Sistemas Operacionais combinam conceitos de praticamente todas as áreas da computação. Seu conhecimento fornece base sólida para construção de sistemas complexos.

Conceitos fundamentais

Os principais conceitos estudados em sistemas operacionais incluem:



Processos

Unidade básica de execução no sistema.



Arquivos

Abstração para armazenamento persistente.



Proteção e Chamadas de Sistema

Segurança e interface com o kernel.



Espaços de Endereçamento

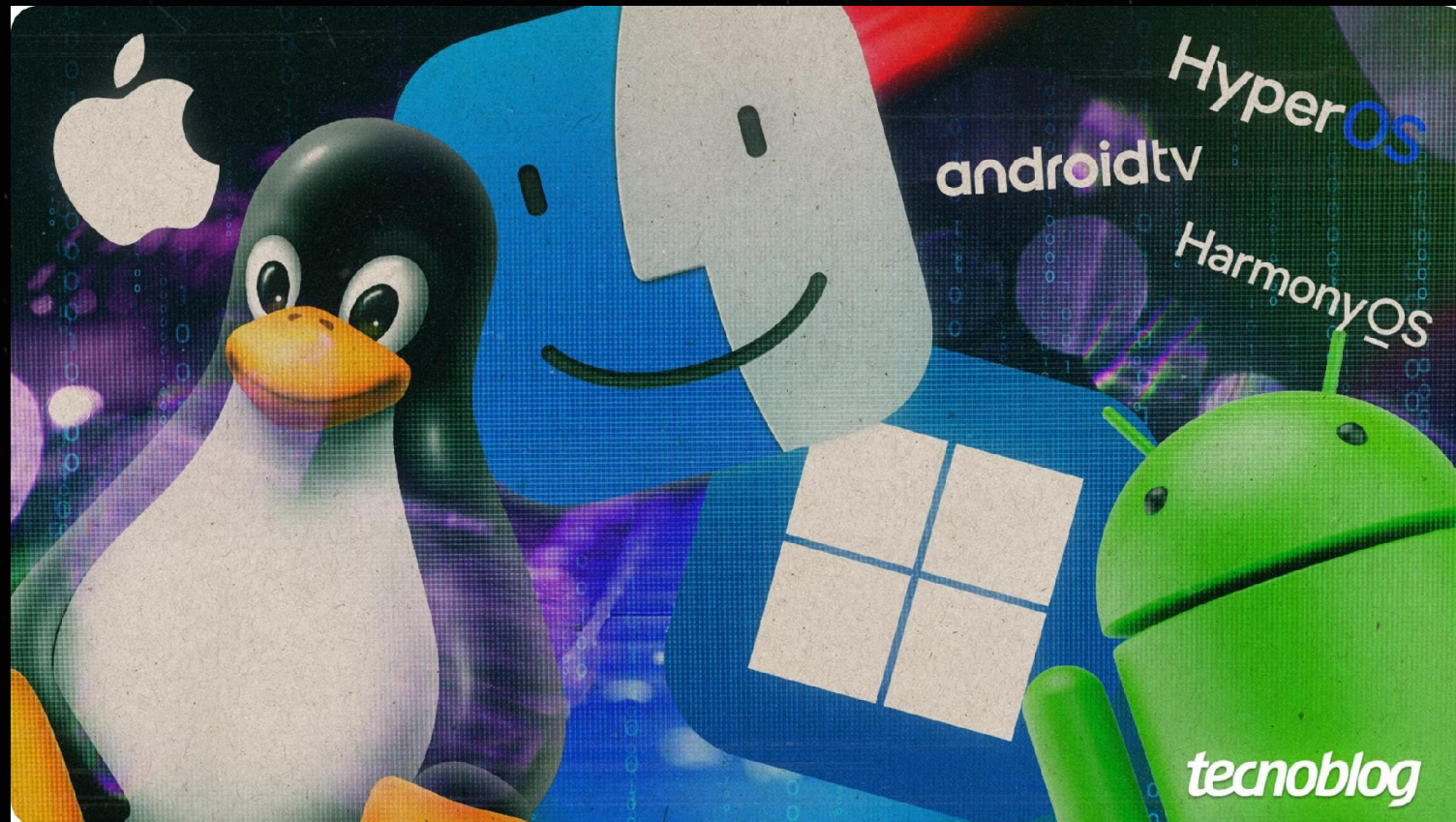
Região de memória de cada processo.



Entrada e Saída

Comunicação com periféricos e dispositivos.

Tipos de Sistemas Operacionais



Diferentes SOs

Existem muitos tipos de sistemas operacionais porque existem muitos tipos de computadores e contextos de uso. O sistema operacional de um servidor bancário não tem as mesmas prioridades de um smartphone, de um sistema embarcado ou de um cartão inteligente.



Grande porte

Alto volume, confiabilidade



Servidores

Disponibilidade e rede



Computadores Pessoais

Usabilidade e compatibilidade



Móveis

Bateria e sensores



Embarcados

Função específica



Tempo real

tempo como um parâmetro-chave.

Ideia central: Cada tipo de SO é projetado para resolver problemas específicos.

Sistemas de computadores de grande porte

Sistemas de mainframes são voltados a computadores de grande porte, geralmente usados em grandes organizações.

Características

- Grande capacidade de entrada e saída
- Processamento de muitos jobs
- Suporte a processamento em lote
- Processamento de transações
- Tempo compartilhado

Exemplos de uso

- Bancos
- Companhias aéreas
- Grandes sistemas corporativos
- Comércio eletrônico em larga escala





IBM ZOS

O **IBM z/OS** é um sistema operacional de 64 bits criado pela IBM para computadores de grande porte conhecidos como mainframes IBM.

<https://www.ibm.com/br-pt/products/zos>

Sistemas operacionais de servidores

São sistemas projetados para oferecer serviços a muitos usuários ou máquinas pela rede.

Exemplos de serviços

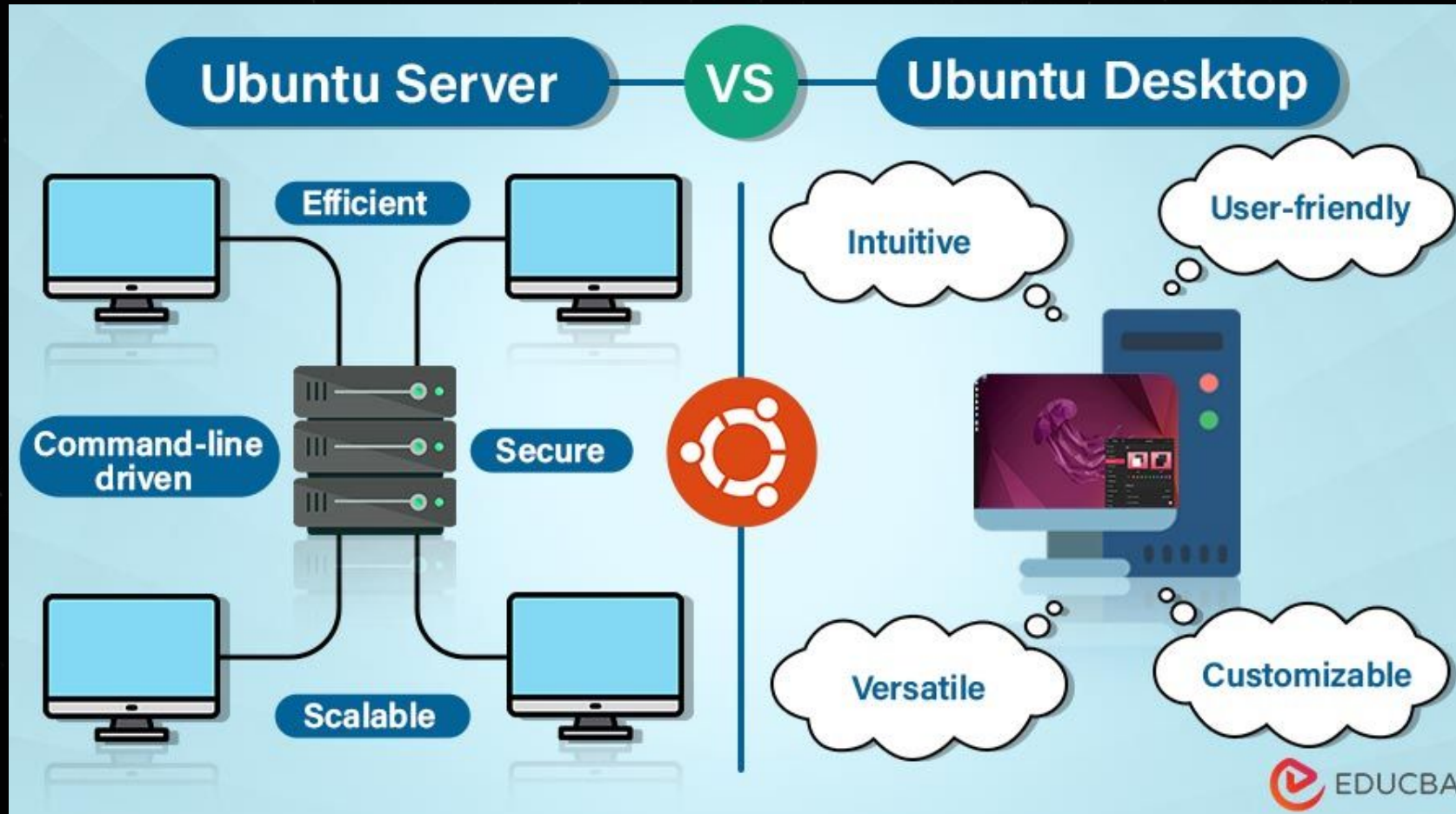
- Páginas web
- Bancos de dados
- Arquivos
- Autenticação
- Aplicações corporativas

Características

- Alta disponibilidade
- Segurança
- Gerenciamento de usuários
- Uso eficiente de CPU, memória, disco e rede



Ubuntu Server



Sistemas operacionais de computadores pessoais

São sistemas voltados ao uso individual, equilibrando desempenho, usabilidade e compatibilidade.



Facilidade de uso

Interface gráfica intuitiva e amigável.



Compatibilidade

Suporte amplo a aplicativos e dispositivos.



Exemplos

Windows, macOS e distribuições Linux para desktop.





Sistemas operacionais de computadores pessoais

São sistemas voltados ao uso individual, equilibrando desempenho, usabilidade e compatibilidade.



Facilidade de uso

Interface gráfica intuitiva e amigável.



Compatibilidade

Suporte amplo a aplicativos e dispositivos.



Exemplos

Windows, macOS e distribuições Linux para desktop.



Sistemas operacionais de dispositivos móveis

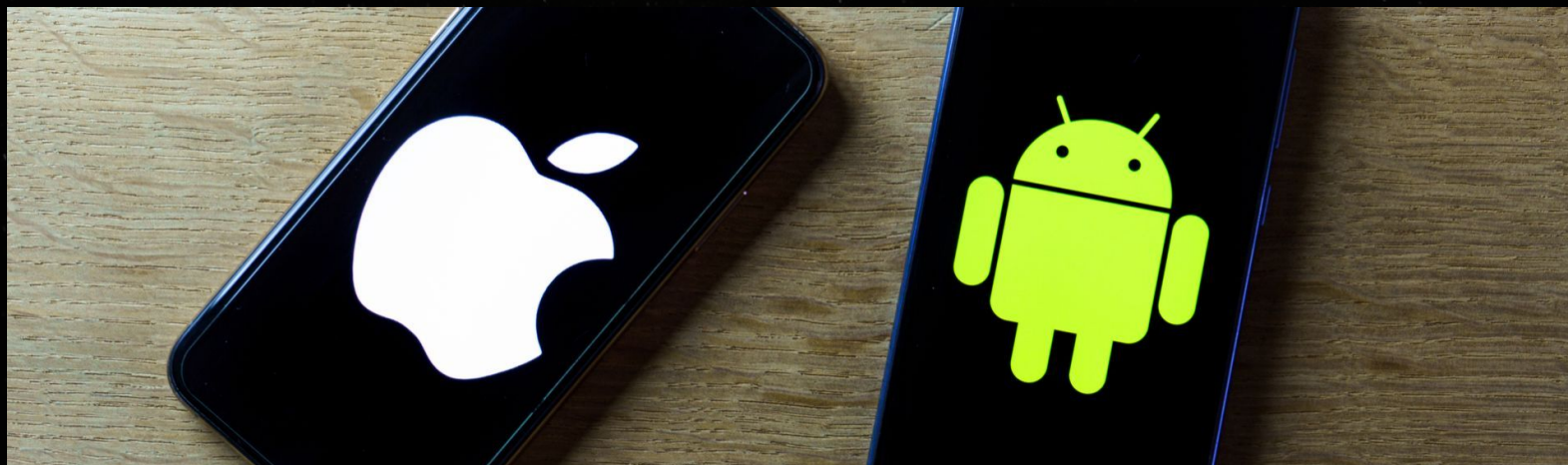
Projetados para smartphones e tablets, com restrições e exigências próprias.

Características

- Interface por toque
- Economia de energia
- Sensores integrados
- Conexão sem fio
- Controle de permissões
- Isolamento entre aplicativos
- Loja de aplicativos

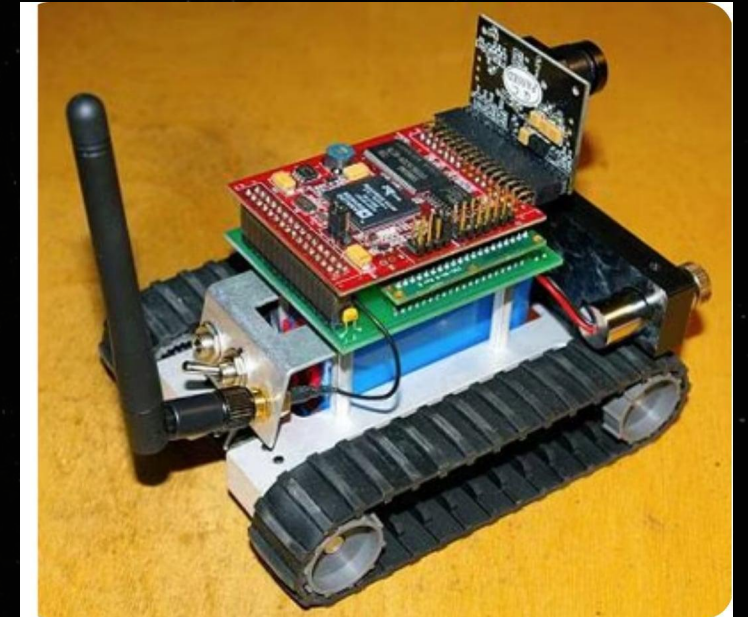
Exemplos

- Android
- iOS



Sistemas embarcados

- Executam dentro de dispositivos específicos
- Geralmente têm função bem definida
- Aparecem em carros, eletrodomésticos, roteadores e equipamentos industriais



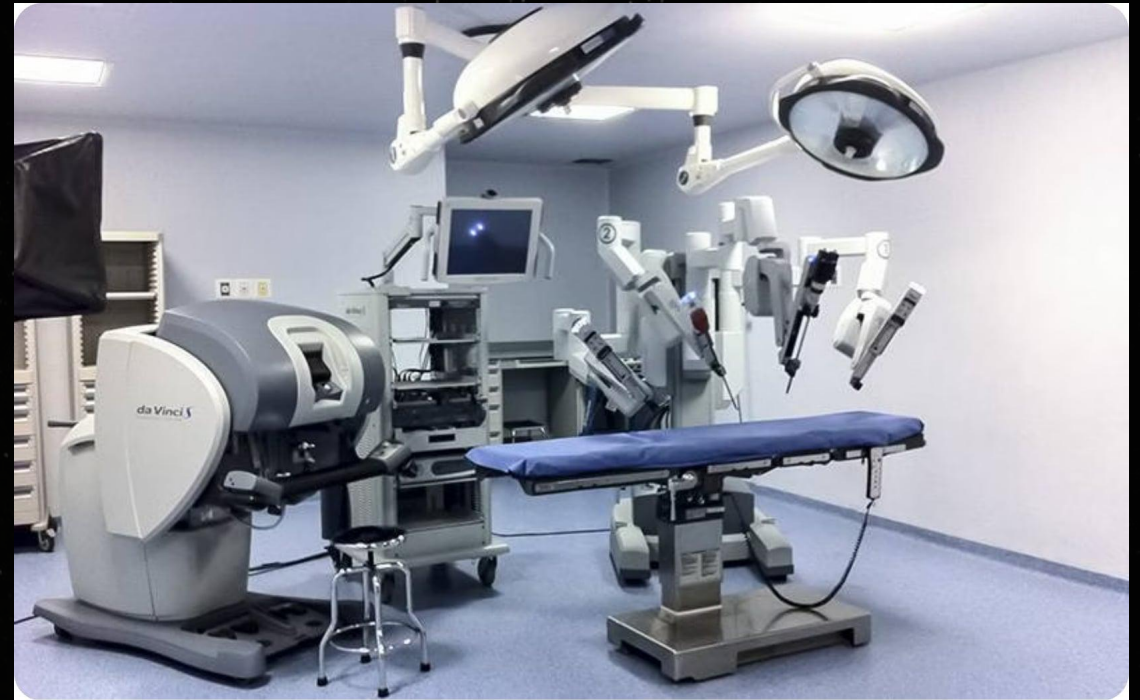
Exemplos de sistemas operacionais embarcados

- eCos.
- Windows CE.
- VxWorks.
- Inferno.
- Free RTOS.
- QNX.
- Esystech X RTOS (nacional).
- VirtuOS (nacional).
- HardHat Linux (Montavista).
- Zeroshell Linux.
- Ångström Linux.
- Palm OS.
- Symbian OS.



Sistemas de **tempo real**

- Usados em controle industrial, aviônica, equipamentos médicos e automação
- Precisam responder dentro de prazos definidos



Obs:Nem todo SO é feito para interação direta com o usuário. Alguns operam quase **invisíveis**.

● kaishinaw@Aws-MacBook-Pro ERC721 % npx hardhat

```
888      888      888 888      888
888      888      888 888      888
888      888      888 888      888
8888888888 8888b. 888d888 .d88888 88888b. 8888b. 888888
888      888      "88b 888P" d88" 888 888 "88b      "88b 888
888      888 .d888888 888      888 888 888 .d888888 888
888      888 888 888 888      Y88b 888 888 888 888 Y88b.
888      888 "Y888888 888      "Y88888 888 888 "Y888888 "Y888
```

 Welcome to Hardhat v2.10.2 

✓ What do you want to do? · Create a JavaScript project

Outras categorias de sistemas operacionais

O "zoológico" dos SOs não se esgota nos tipos mais comuns. Também existem:



Nós Sensores

Sistemas operacionais para redes de sensores com recursos extremamente limitados.



Cartões Inteligentes

SOs de cartões com CPU própria, memória restrita e alta segurança.



Sistemas Distribuídos

Múltiplas máquinas trabalhando como um sistema coeso.



Dispositivos Especializados

SOs criados para finalidades muito específicas de hardware ou contexto.

Ideia central: Quanto mais diverso o hardware e o contexto de uso, mais diverso será o projeto do sistema operacional.



Chamadas de Sistemas

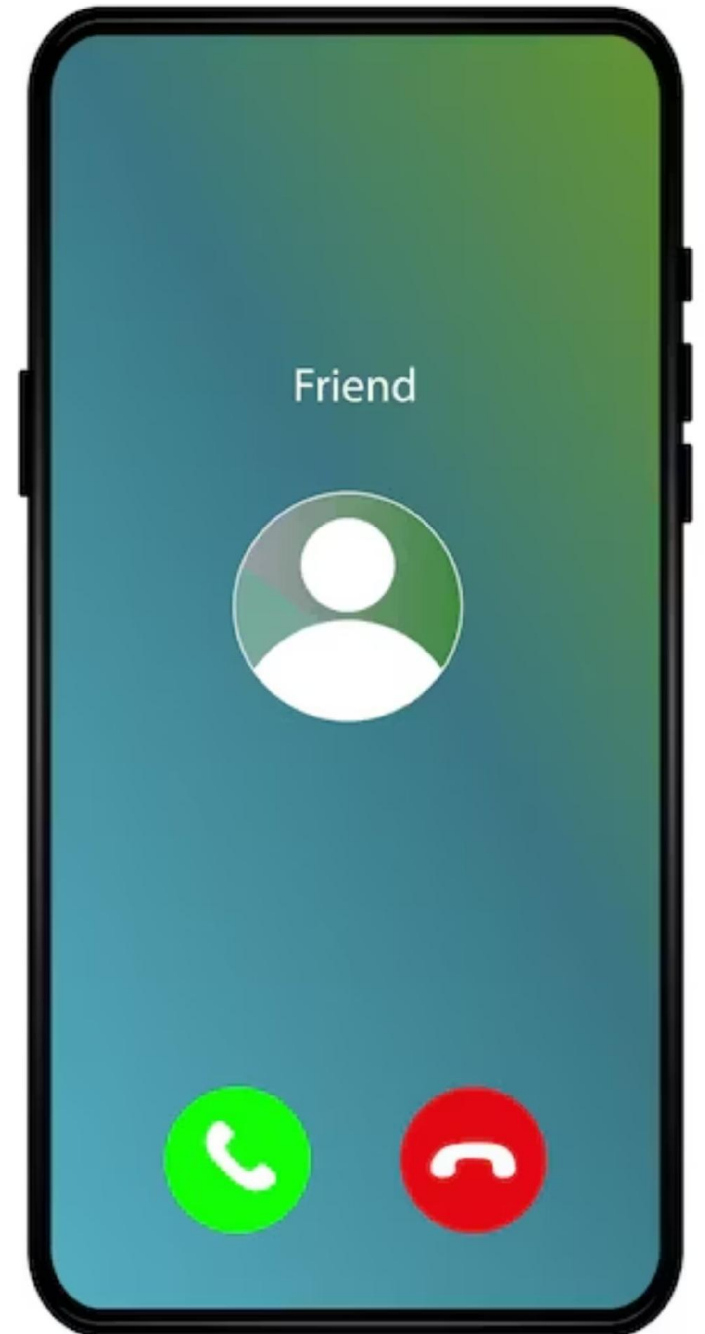
Chamadas de sistema

Uma chamada de sistema(system calls) é uma rotina que permite que um aplicativo de usuário solicite ações que requerem privilégios especiais. A inclusão de chamadas do sistema é uma das várias formas de ampliar as funções fornecidas pelo kernel.

Chamadas de sistema

Chamadas de sistema são a **porta de entrada para os serviços do kernel**. Elas permitem que programas solicitem ações ao sistema operacional, como:

- 1 Criar processos e gerenciar execução
- 2 Abrir, ler e escrever arquivos
- 3 Acessar dispositivos e obter informações
- 4 Encerrar execução de programas



Por que não acessar o hardware **diretamente**?

Programas de usuário normalmente não podem acessar diretamente recursos sensíveis do sistema.



Segurança

Impede que programas maliciosos comprometam o sistema.



Estabilidade

Evita que falhas em um programa derrubem o sistema inteiro.



Proteção entre Processos

Garante que um processo não interfira nos dados de outro.



Controle de Permissões

Organiza e controla quem pode acessar o quê no sistema.



Assim, o programa pede ao sistema operacional por meio de **chamadas de sistema**.

Troca de Contexto em Chamadas de Sistema

Quando uma aplicação faz uma chamada de sistema, ocorre uma transição fundamental no processador para garantir a segurança e a integridade do sistema.

Modo Usuário para Modo Núcleo

O processador muda de um estado não privilegiado para um privilegiado, permitindo acesso a recursos críticos.

Salvamento de Contexto

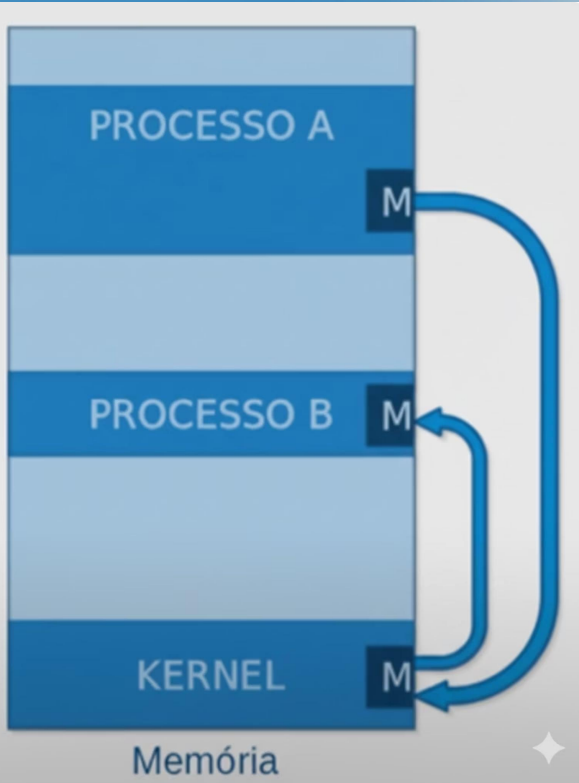
O estado atual (contexto) do programa de usuário é salvo para que possa ser restaurado posteriormente.

Execução da Chamada

O kernel executa a operação solicitada, utilizando suas permissões elevadas.

Retorno ao Modo Usuário

Após a conclusão, o contexto do usuário é restaurado e o processador volta ao modo não privilegiado.



Categorias de chamadas de sistema

As chamadas de sistema podem ser agrupadas em categorias por finalidade, representando o conjunto de serviços oferecidos pelo SO:

Controle de Processos

Criar processo, encerrar execução, aguardar evento.

Manipulação de Arquivos

Abrir, fechar, ler, escrever e renomear arquivos.

Manipulação de Dispositivos

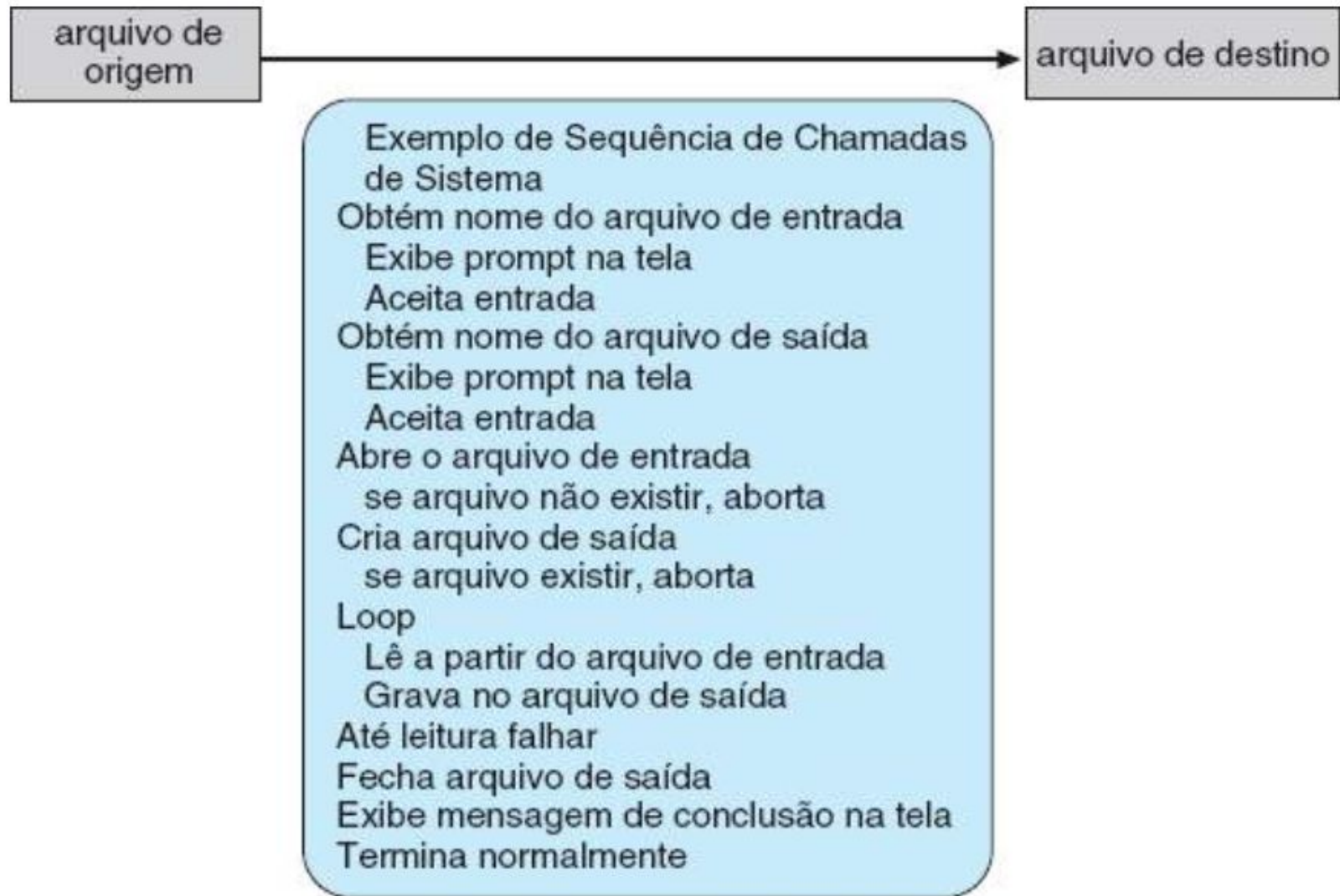
Ler e escrever em dispositivos de hardware.

Manutenção de Informações

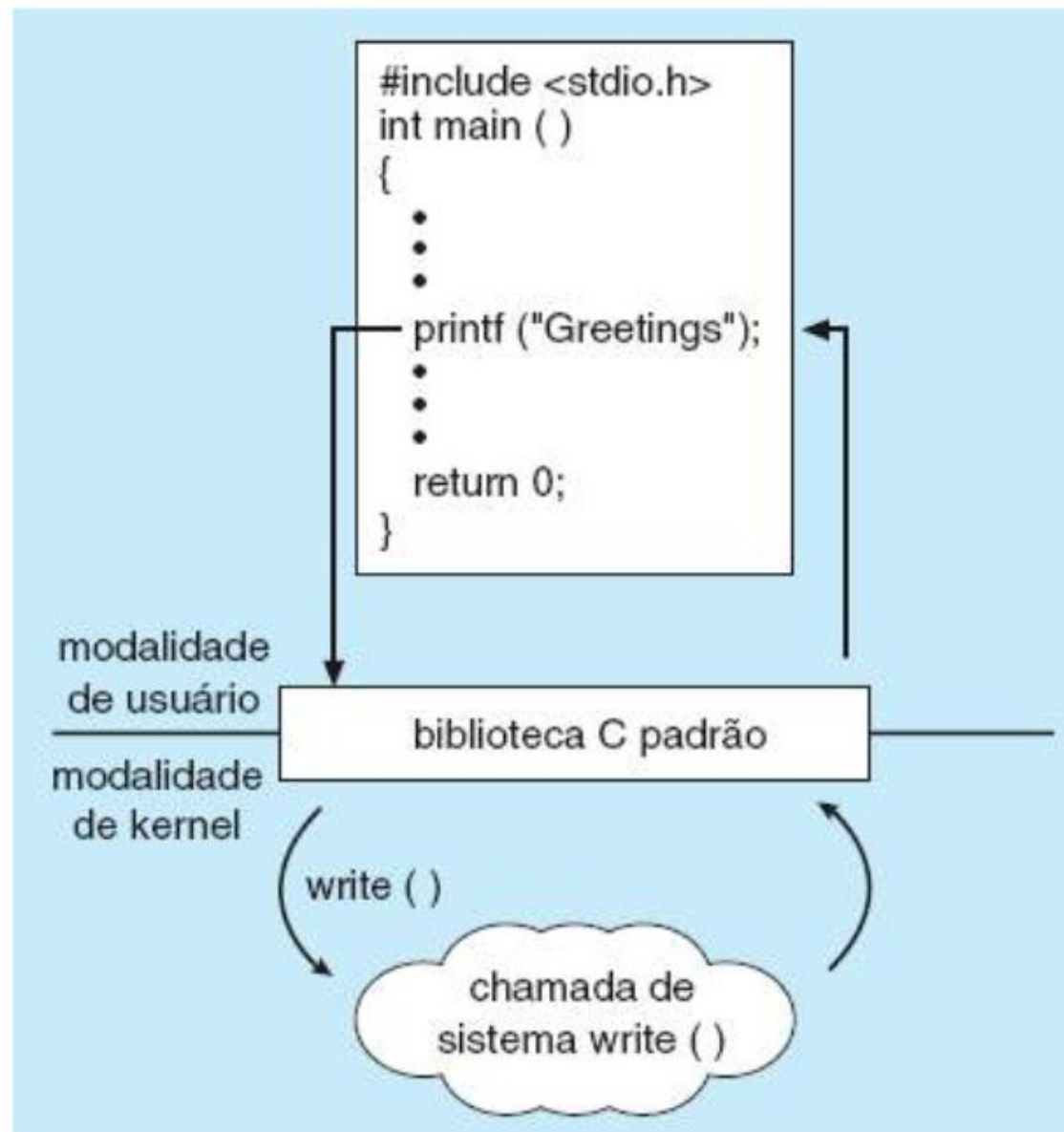
Obter hora do sistema, informações de processos e recursos.

Comunicação e Proteção

Enviar mensagens, criar canais e alterar permissões.



Exemplo de como as chamadas de sistema são usadas.



Estruturas de Sistemas Operacionais

Por que estudar a estrutura do SO?

Sistemas operacionais são grandes e complexos. A estrutura interna define:

- Como os componentes se organizam internamente
- Onde os serviços executam: no kernel ou fora dele
- Como os módulos se comunicam entre si
- Quão fácil é manter, atualizar e depurar o sistema
- Quão seguro e eficiente o sistema pode ser



Estrutura simples

Muitos sistemas operacionais não têm estruturas bem definidas. Frequentemente, tais sistemas começam como sistemas pequenos, simples e limitados e, então, crescem para além de seu escopo original. O MS-DOS é um exemplo desse tipo de sistema. Ele foi originalmente projetado e implementado por algumas pessoas que não tinham ideia de que se tornaria tão popular. **Ele foi escrito para fornecer o máximo de funcionalidade no menor espaço e, portanto, não foi dividido cuidadosamente em módulos.**



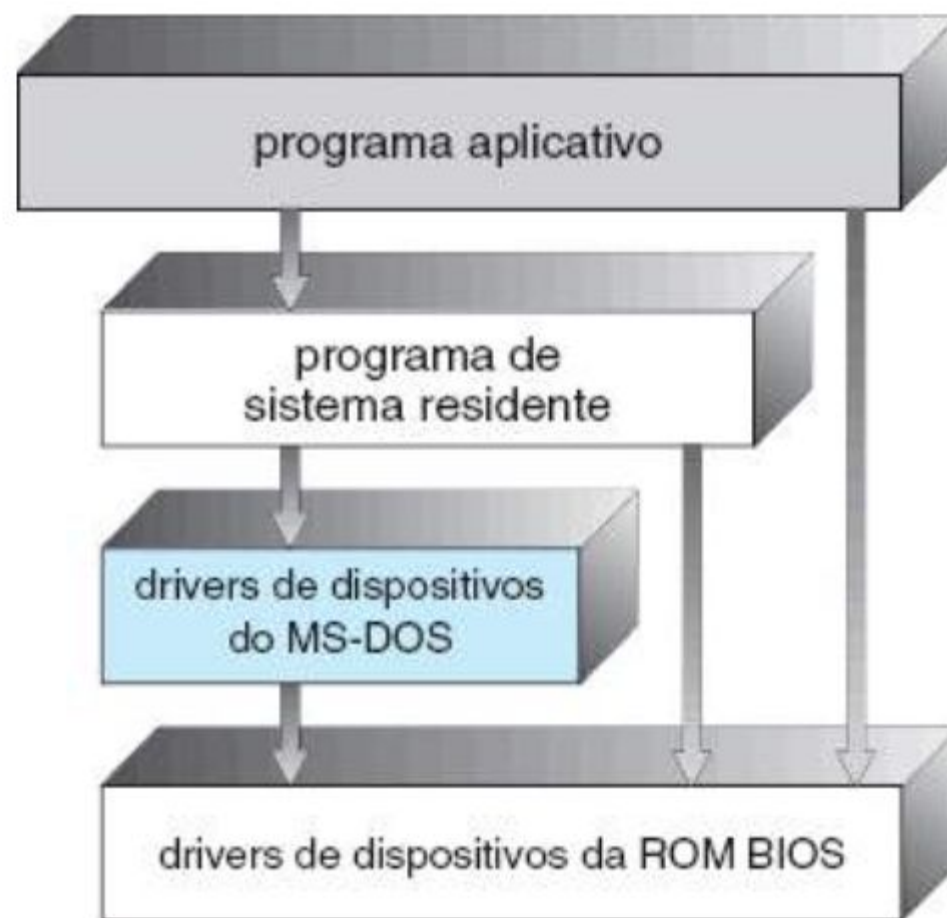


Figura 2.11 Estrutura em camadas do MS-DOS.

Abordagem em camadas

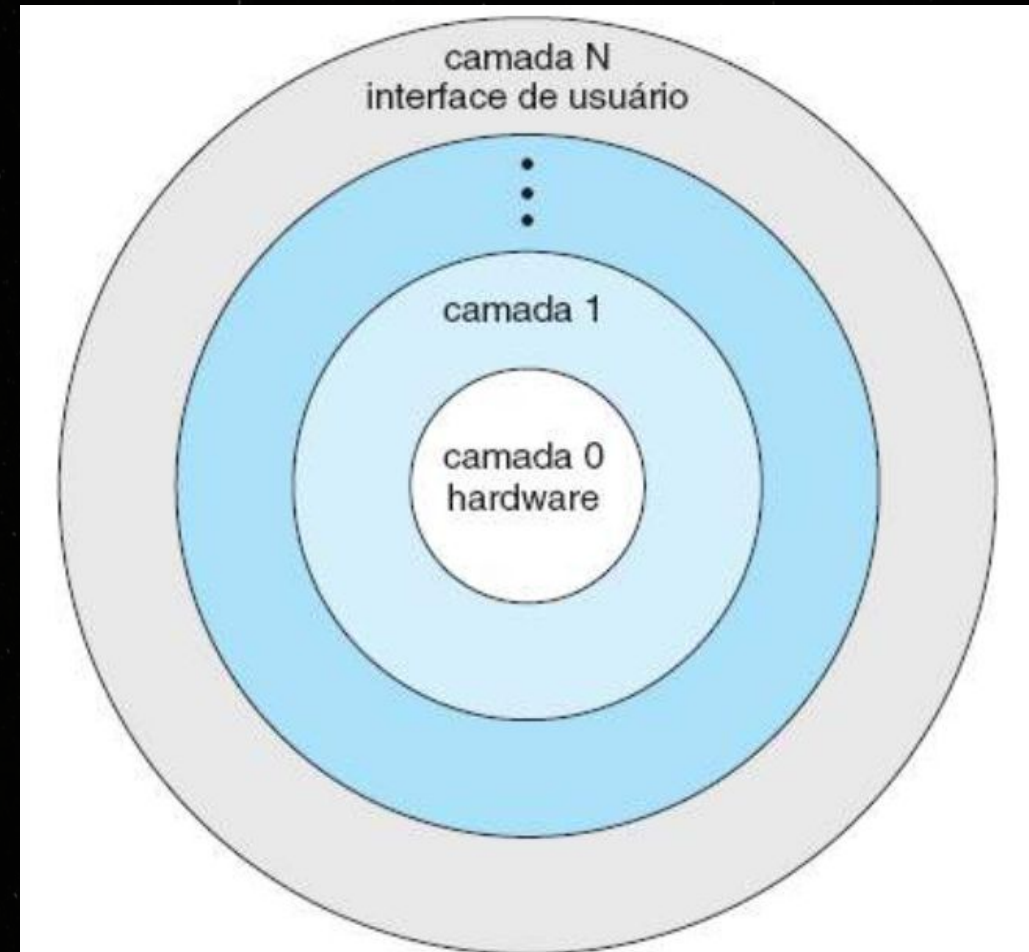
A arquitetura em camadas organiza o SO em níveis hierárquicos bem definidos.

- Cada camada usa serviços da camada inferior.
- Cada camada oferece serviços para a camada superior.
- Facilita teste, manutenção e segurança.

Camada	Função
5	O operador
4	Programas de usuário
3	Gerenciamento de entrada/saída
2	Comunicação operador–processo
1	Memória e gerenciamento de tambor
0	Alocação do processador e multiprogramação

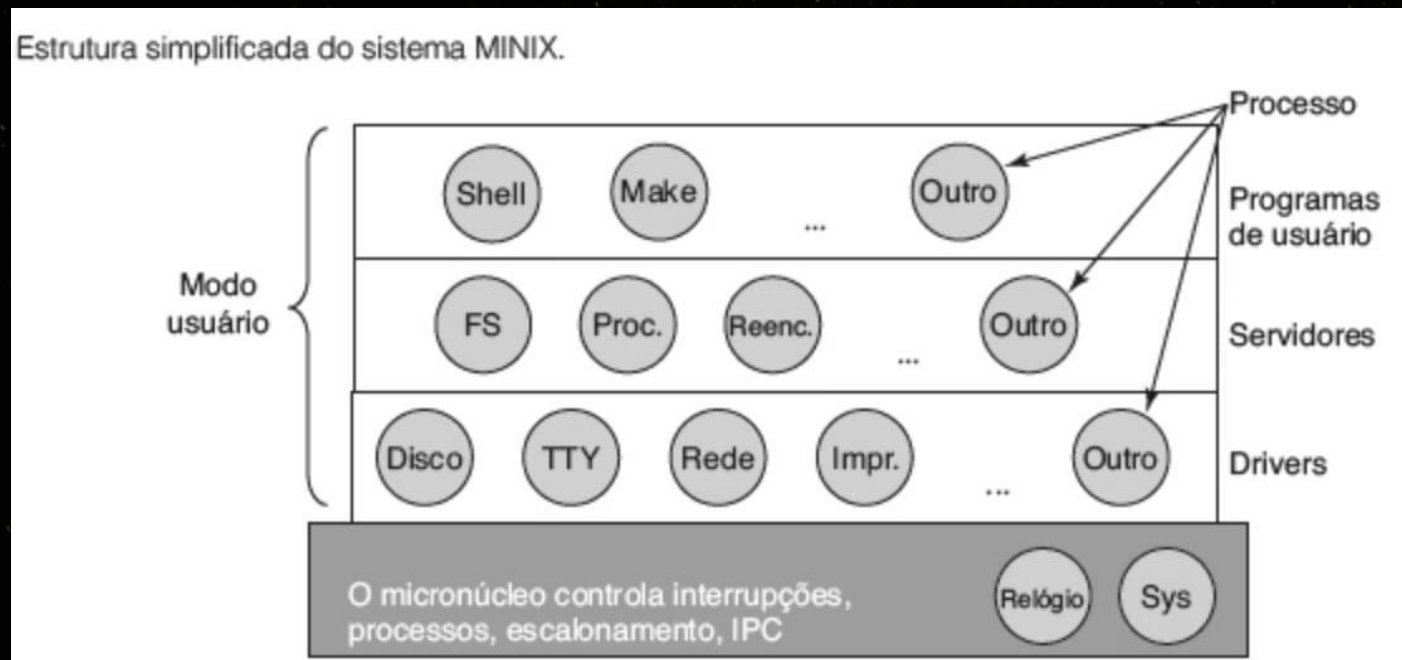
O primeiro sistema construído dessa maneira foi o sistema THE desenvolvido na Technische Hogeschool Eindhoven na Holanda por E. W. Dijkstra (1968) e seus estudantes.

Quais as desvantagens
da abordagem em
camadas?



Micronúcleos

- Mantém no kernel apenas funções essenciais;
- Move vários serviços para o espaço de usuário;
- Busca maior modularidade, isolamento, confiabilidade e extensibilidade;
- Realiza troca de mensagens entre processos.



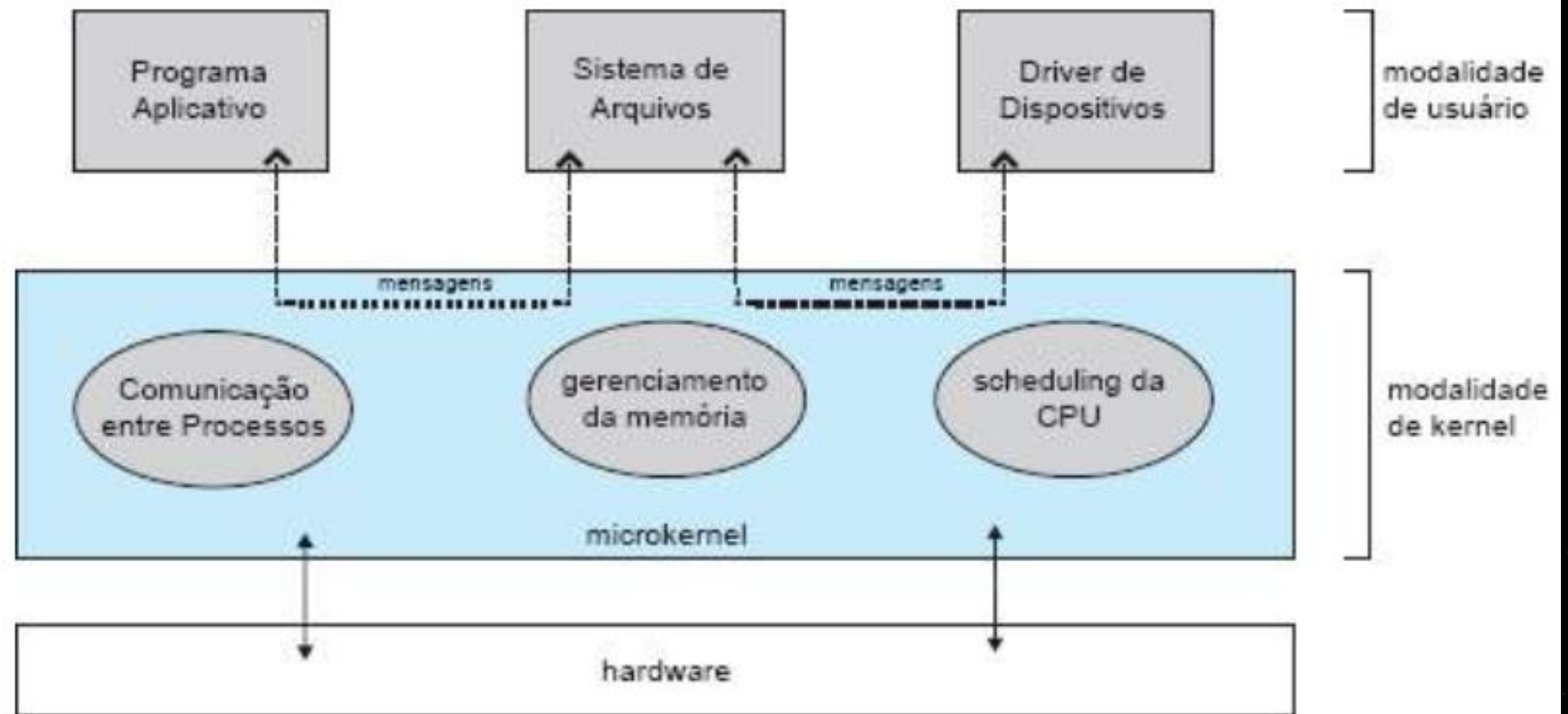


Figura 2.14 Arquitetura de um microkernel típico.

**Quais as desvantagens dos
microkernels?**

Caso Windows NT

Infelizmente, o desempenho dos microkernels pode ser afetado pelo aumento do **overhead de funções de sistema**. Considere a história do Windows NT. A primeira versão tinha uma organização de microkernel em camadas. O desempenho dessa versão era baixo, se comparado ao do Windows 95. O Windows NT 4.0 corrigiu parcialmente o problema de desempenho movendo camadas do espaço do usuário para o espaço do kernel e integrando-as mais fortemente. Quando o Windows XP foi projetado, a arquitetura Windows tinha se tornado mais monolítica do que de microkernel.

Estrutura monolítica

Em sistemas monolíticos, grande parte do sistema operacional executa no núcleo (modo privilegiado), com chamadas diretas entre componentes internos.

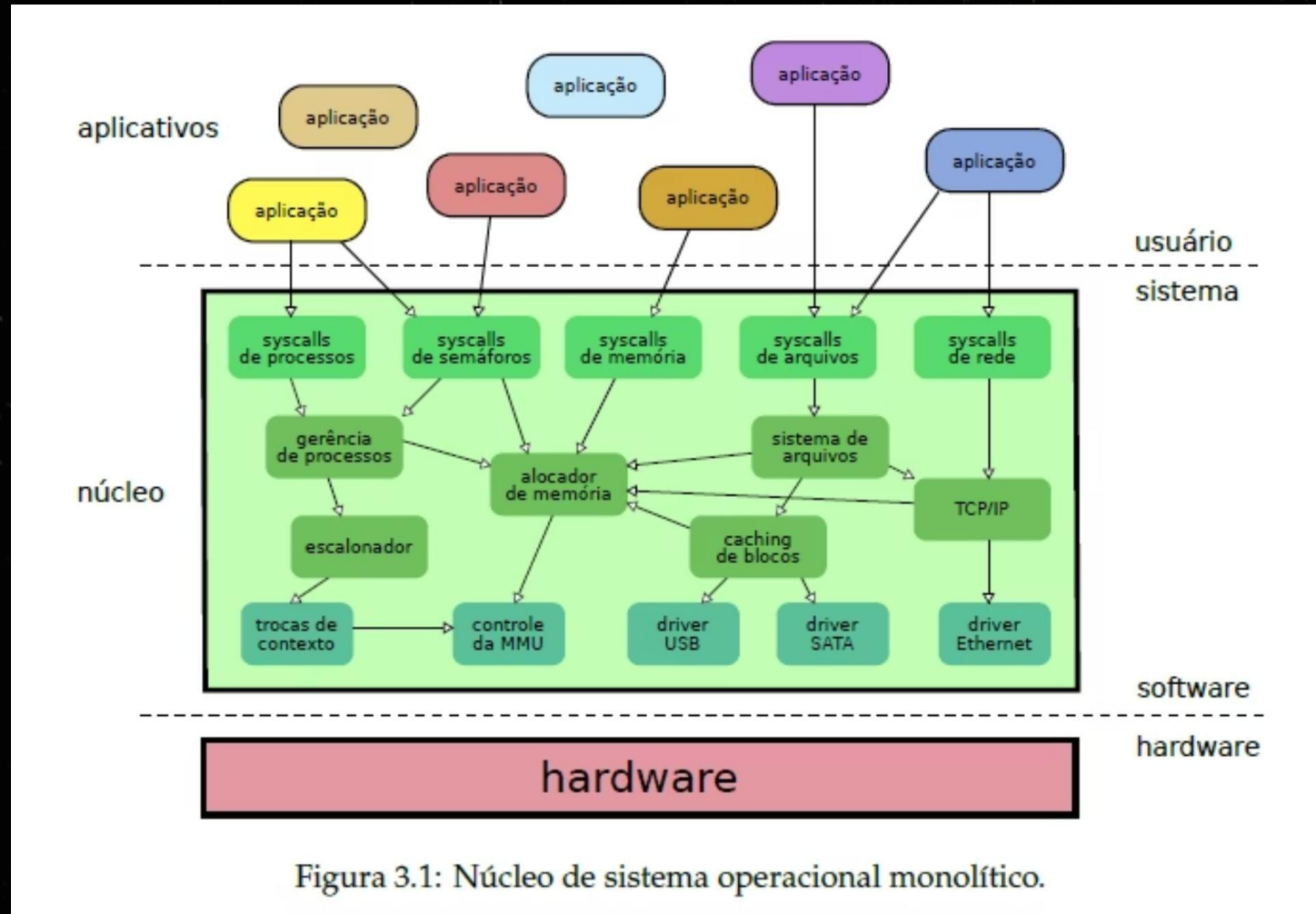
Características

- Muitos serviços dentro do kernel
- Chamadas diretas entre componentes internos
- Alto desempenho
- Facilidade

Serviços no núcleo

- Gerenciamento de processos
- Memória
- Sistema de arquivos
- Drivers
- Rede

Estrutura monolítica



**Quais as desvantagens das estruturas
monolíticas?**

Módulos

O kernel tem um conjunto de componentes nucleares e vincula serviços adicionais por meio de módulos, tanto em tempo de inicialização quanto em tempo de execução. Esse tipo de projeto é comum em implementações modernas do UNIX e, também, do Solaris, do Linux e do Mac OS X, assim como do Windows.

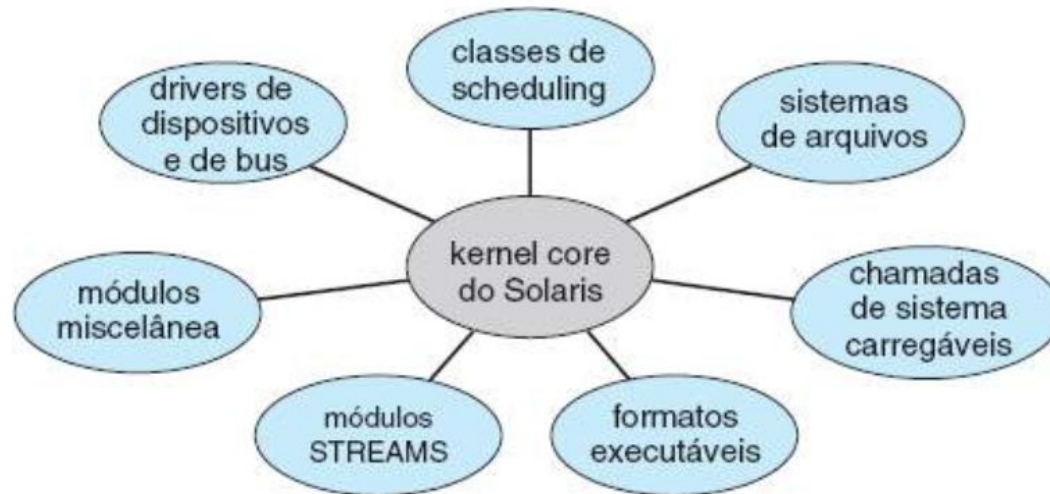


Figura 2.15 Módulos carregáveis do Solaris.

Módulos

- O resultado final lembra um sistema em camadas em que cada seção do kernel tem interfaces definidas e protegidas, porém, ele é mais flexível do que um sistema em camadas porque um módulo pode chamar qualquer outro módulo.
- A abordagem também é semelhante à abordagem de microkernel, já que o módulo principal tem apenas funções nucleares e o conhecimento de como carregar e se comunicar com outros módulos; no entanto, é mais eficiente porque os módulos não precisam invocar a transmissão de mensagens para se comunicarem.

Híbridos

Na prática, muito poucos sistemas operacionais adotam uma estrutura única rigidamente definida. Em vez disso, **eles combinam diferentes estruturas, resultando em sistemas híbridos que resolvem problemas de desempenho, segurança e usabilidade.**

- Por exemplo, tanto o Linux quanto o Solaris são monolíticos porque o desempenho é muito mais eficiente quando o sistema operacional ocupa um único espaço de endereçamento. No entanto, eles também são modulares para que novas funcionalidades possam ser adicionadas ao kernel dinamicamente.
- O Windows também é amplamente monolítico (mais uma vez por questões de desempenho, principalmente), mas retém certo comportamento típico de sistemas de microkernel, inclusive fornecendo suporte a subsistemas separados. Os sistemas Windows também fornecem suporte para módulos do kernel carregáveis dinamicamente.

Exemplo: iOS

- **Cocoa Touch** é uma API para Objective-C que fornece várias estruturas para o desenvolvimento de aplicativos que executam em dispositivos iOS.
- A **camada de serviços** de mídia fornece serviços para ambientes gráficos, áudio e vídeo
- A **camada de serviços nucleares** fornece uma variedade de recursos, incluindo suporte à computação em nuvem e a bancos de dados.



Exemplo: Android

Android é semelhante ao iOS por ser uma pilha de softwares em camadas que fornece um rico conjunto de estruturas para o desenvolvimento de aplicativos móveis. **Na base dessa pilha de softwares, está o kernel Linux**, embora ele tenha sido modificado pela Google e esteja, atualmente, fora da distribuição normal de versões do Linux.

O Linux é usado, principalmente, no suporte de hardware a processos, memória e drivers de dispositivos e tem sido expandido para incluir gerenciamento de energia.

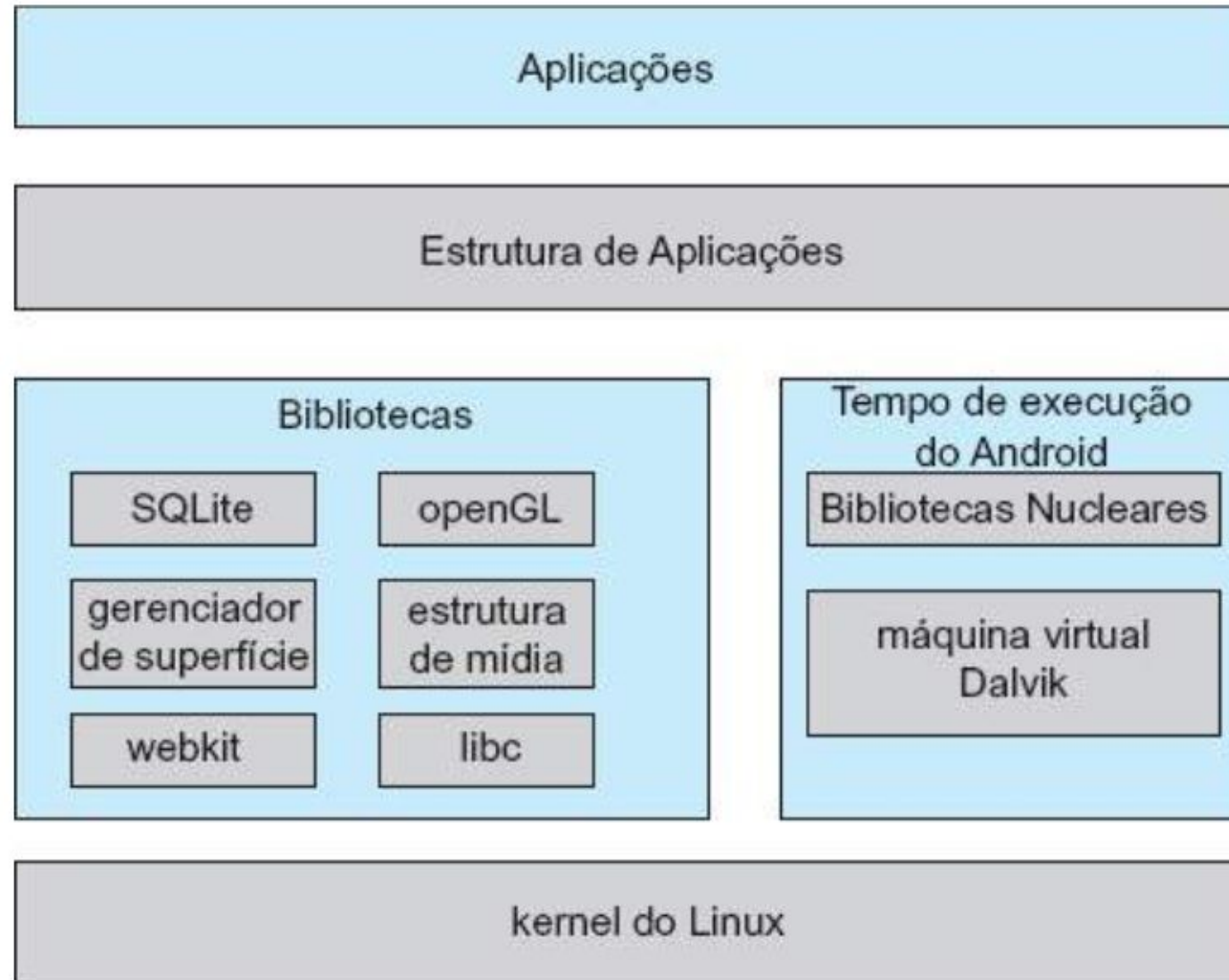
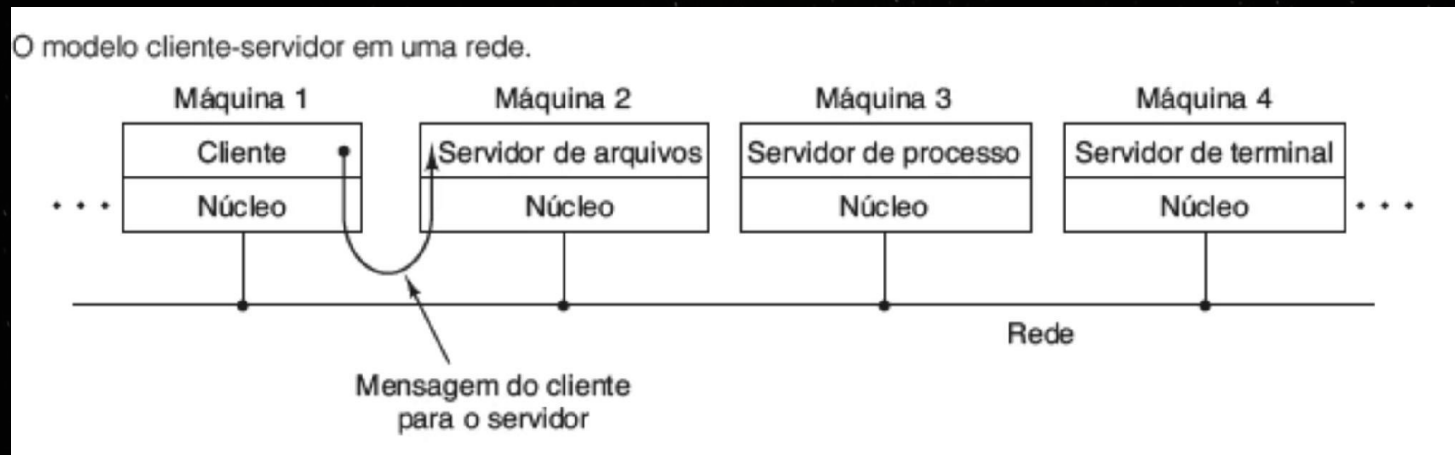


Figura 2.18 Arquitetura do Android da Google.

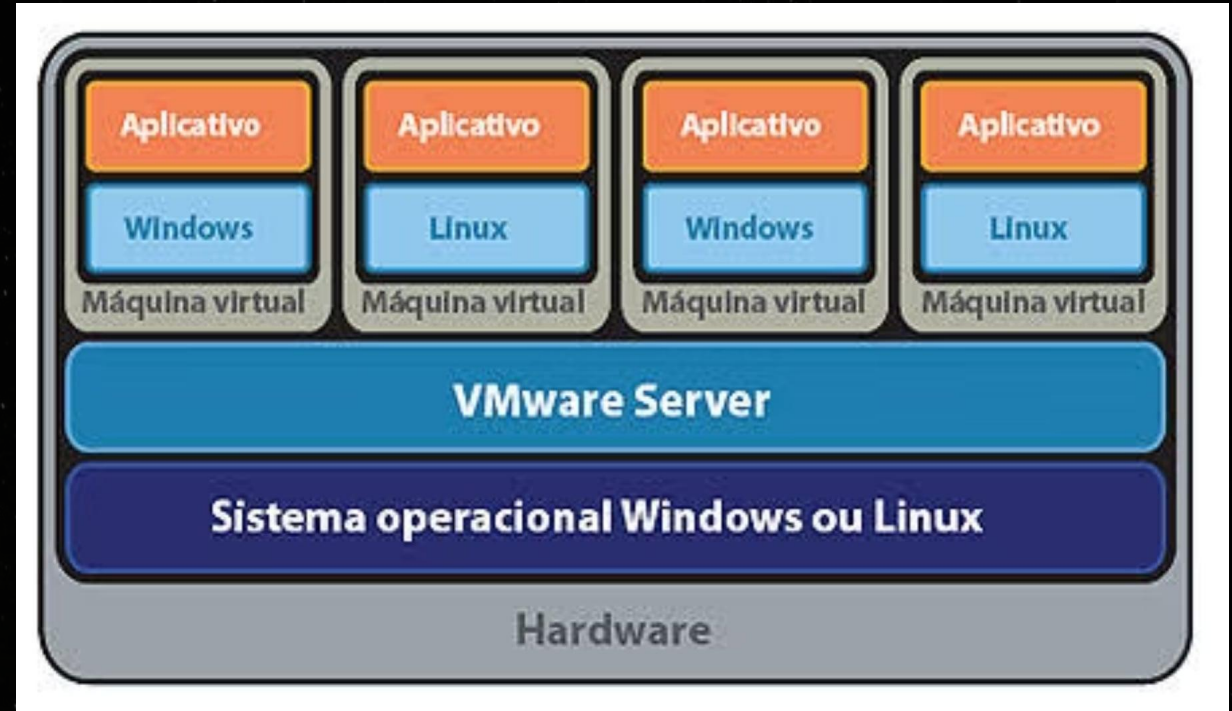
Estrutura cliente-servidor

- Processos clientes solicitam serviços
- Servidores respondem às solicitações
- Comunicação ocorre por mensagens



Máquinas virtuais

- Criam abstrações de hardware
- Permitem executar sistemas ou ambientes isolados
- Úteis para testes, servidores, nuvem e segurança.





Moral da história: Arquiteturas diferentes buscam equilíbrios diferentes entre **controle, desempenho, segurança e flexibilidade**.

Referências bibliográficas

TANENBAUM, Andrew S.; BOS, Herbert.

Sistemas Operacionais Modernos. 4. ed. São Paulo: Pearson, 2016.

SILBERSCHATZ, Abraham; GALVIN, Peter B.; GAGNE, Greg.

Fundamentos de Sistemas Operacionais. 9. ed. Rio de Janeiro: LTC, 2015.

MAZIERO, Carlos A.

Sistemas Operacionais: Conceitos e Mecanismos. Curitiba: UFPR, 2019.

A terminal window is displayed on a computer monitor. The monitor has a light-colored, slightly textured bezel. The terminal window itself is a dark rectangle with a thin border. Inside the terminal, the text 'bccunifal@root: ~\$ exit' is written in a white, monospaced font. The background of the entire image shows a desk with a keyboard and some papers, though they are out of focus.

`bccunifal@root: ~$ exit`